The Evidence Peer-Review System

Engineering an open, tamper-evident,
capture-resistant, evolving consensus

The Disclosure Platform

May 2026

Abstract. This is an engineering paper. It specifies the system that turns the philosophy of *Disclosure: A Labour of Love* into enforceable mechanism. A single smart contract, `EvidenceConsensus`, deployed on BNB Smart Chain, is the sole source of truth for the peer set, the Pillar → Topic taxonomy, and the lifecycle of every record; a read-only sidecar, `EvidenceConsensusLens`, serves aggregate views without holding any authority. A Supabase database is a fast, rebuildable *projection* of the chain, kept in step by an indexer that joins rows to on-chain events by deterministic hashes. We specify how a record's content is hashed so any later tampering is detectable; how the taxonomy is grown by peer consensus and why a node is never empty; the *binding* — an (evidence × topic) pair — as the atomic unit of review; the eight-state lifecycle a binding moves through; the bounded submission queue that keeps open submission from outrunning review capacity; the threshold mathematics that scales with the active peer set and makes "Canon" a true majority; how vote outcomes are made order-independent via peer-count snapshots; the time-boxed challenge procedure; the administrator-free peer registry; the BFT-style capture boundary and the peer *force-renounce* backstop against a captured owner; peer-liveness garbage collection; and the EIP-712 attestation scheme that makes every vote a signed, human-readable, independently verifiable artefact. We close by mapping each mechanism back to the principle it secures.

Contents

1 Architecture: a chain of truth and a cache of speed

The system is two layers with a strict division of authority, plus a read-only sidecar that holds none.

Layer 1 — the chain of truth. A single Solidity contract, `EvidenceConsensus`, on BNB Smart Chain (BSC), holds everything that must be authoritative and tamper-resistant: the set of verified peers, the taxonomy of pillars and topics, the state and vote tallies of every record, and the time windows that govern them. Nothing is canon unless the chain says so. Because the chain is public and append-only, the entire deliberative history is preserved end to end and cannot be silently rewritten.

The read sidecar. As the core grew — a submission queue and on-chain peer garbage collection were added — its deployed bytecode approached the EIP-170 limit of 24,576 bytes. Pure aggregation views (assembling the peer list, the nominee list, and the pending-proposal list with their tallies) were therefore moved into a separate contract, `EvidenceConsensusLens`. The Lens holds *no storage*, has *no privileges*, and reconstructs every aggregate purely from the core's public state, so it carries zero consensus risk: the core remains the sole writer and the sole source of truth. Read clients call the Lens for these list views; everything that mutates state goes to the core.

Layer 2 — the cache of speed. A Supabase (Postgres) database holds a *projection* of the chain: the same records and taxonomy, shaped for fast reading, searching, and pagination. The cache is never the authority. It is reconciled to the chain by an indexer edge function, `chain-indexer-evidence`, which reads chain events and joins them to off-chain rows by deterministic hashes (a binding's row is matched to its on-chain event by `binding_hash`; a taxonomy row by `node_hash`). The whole database is built from a consolidated base migration plus a few additive follow-ons, with *zero seed data* — it can be rebuilt from an empty Postgres instance and re-synchronised from the chain.

The frontend. A Vite multi-page React application reads from the cache for speed and writes to the contract through the reader's own wallet (MetaMask). The heavy chain library (`ethers` v6) is loaded lazily, so a visitor who only reads the archive never pays its cost. Reads can optionally use a configured read RPC; writes always go through the user's wallet, so the user — never the platform — signs and pays for every state change they cause.

> **Why this split.** Putting authority on-chain makes the record tamper-evident and ownerless; putting presentation off-chain makes it fast and pleasant; putting list aggregation in a privilege-free sidecar keeps the core small enough to deploy without weakening it. Keeping the cache strictly subordinate — rebuildable, never authoritative — means a compromised or lost database costs convenience, not truth. The truth is whatever the contract can prove.

2 The record and content hashing

A piece of evidence has one canonical identity and one content fingerprint. The identity is a UUID, mapped one-to-one to a `bytes32` for on-chain use. The fingerprint is a keccak-256 hash over the record's *content fields only*:

```
contentHash = keccak256( utf8( canonicalJSON {
    title, source, year, excerpt, link, tier
} ) )
```

The canonical JSON trims each string field and coerces `tier` to a number, so the hash is reproducible byte-for-byte by any party. Two properties of this design matter.

The topic is deliberately excluded from the hash. A record's fingerprint binds *what the evidence is*, not *where it is filed*. This is what lets the same evidence be cross-listed under many topics without ever being re-hashed — one piece of evidence, one `contentHash`, many filings.

The hash is computed identically in three places. The frontend computes it when a record is submitted; the `verify-attestation` edge function recomputes it when a vote is verified; and the `audit-content-hash` edge function recomputes it on a schedule across the archive. All three must produce identical bytes. If the content stored off-chain is ever altered, its recomputed hash no longer matches the hash committed on-chain, and the discrepancy is recorded as a tamper alert. Tampering is not prevented by trust; it is made *detectable* by arithmetic.

3 The Pillar–Topic taxonomy

Evidence is not filed into a flat list. It is filed into a two-level taxonomy that the peers themselves grow: **pillars** (top-level domains) and, beneath each pillar, **topics**. The archive grows *wider* as pillars are added and *deeper* as topics are added, and both kinds of growth happen only by peer consensus.

Every taxonomy node has a deterministic on-chain identity and an off-chain metadata commitment:

```
nodeId = keccak256( utf8( slug ) )
metaHash = keccak256( utf8( canonicalJSON {
    kind, slug, parent, title, blurb, tag
} ) )
```

Because the id is the hash of the slug, the indexer can join an off-chain taxonomy row to its on-chain node by `node_hash` without any shared database key. Both commitments are tamper-audited exactly as evidence content is (Section 2): the `audit-content-hash` job recomputes every pillar/topic's `node_hash` and `metaHash` on a schedule and raises a tamper alert if the

off-chain row has drifted from what the peers committed on-chain, so a silently edited taxonomy node is as detectable as a silently edited record.

3.1 A taxonomy node is never empty

A central rule prevents the archive from filling with empty scaffolding: *a node is never proposed alone.*

- A **pillar** is proposed bundled with its first **topic** *and* a founding piece of **evidence**.
- A **topic** is proposed bundled with a founding piece of **evidence**.

The whole bundle rides on a single endorsement gate. The relevant contract calls carry the entire bundle at once:

```
proposePillar(id, metaHash, topicId, topicMetaHash,
    evidenceId, tier, contentHash)
proposeTopic(id, parentPillar, metaHash,
    evidenceId, tier, contentHash)
```

A proposal is then endorsed by other peers via `endorseNode(id)` (the proposer counts as endorsement number one). When endorsements reach the bundle threshold (Section 7), the node ratifies. For a pillar, its bundled founding topic ratifies in the same act, and while the pillar is still pending it *reserves* both its bundled topic id and its bundled evidence id so nothing else can claim them. Critically, the founding evidence's (evidence × topic) binding is canonized *atomically* with ratification — it goes straight to Canon, because the taxonomy endorsement *is* the consensus for that founding record.

> **Bundling is not a cheaper way in.** The founding evidence is not exempt from review-grade consensus. The bundle gate is max(taxonomyThreshold, canonizeThreshold(tier)), so smuggling a record in as a thin "new topic" is never cheaper than the normal review path. A pillar with no topic, or a topic with no evidence, is an opinion about how the archive *should* be organised with nothing yet to organise. Bundling forces every act of structure to arrive already carrying content, so the taxonomy can only grow where there is real evidence to fill it. Further evidence added later to a ratified topic (`submitEvidence` / `fileBinding`) passes through ordinary tier-based review.

A proposal that never reaches its gate is not stranded forever: after the 30-day `PROPOSAL_WINDOW` anyone may call `lapseProposal(id)`, which garbage-collects the node and frees its reserved topic and evidence ids for a fresh attempt. Each peer is also capped at a small number of outstanding (still-proposed) proposals, the on-chain analogue of the off-chain `tax_insert:` throttle, so no single peer can flood the proposal queue.

4 Bindings: the atomic unit of review

The system does not review "evidence" in the abstract. It reviews a **binding**: a specific (evidence × topic) pair. Each binding is an independent voting unit with its own lifecycle state and its own tallies, and its own deterministic id:

```
bindingId = keccak256( abi.encode( evidenceId, topicId ) )
```

This is the structural consequence of "one record, many filings." A single piece of evidence (one `contentHash`) may be filed under any number of topics, and each filing is judged on its own merits. The same record can be Canon under one topic and Expelled under another, because relevance is contextual: a document may be excellent evidence for one subject and irrelevant to another.

Two contract calls create bindings. `submitEvidence(id, tier, topicId, contentHash)` registers a new record and opens its first binding. `fileBinding(id, topicId)` cross-lists an already-registered record under a further ratified topic, opening a new independent binding. Every subsequent vote or challenge call is addressed to a binding by its `(id, topicId)` pair.

> **Open submission.** `submitEvidence` and `fileBinding` are callable by any wallet; filing evidence is not gated. What is gated is *judgement* — only verified peers can vote a binding through its lifecycle. This is the contract-level expression of "anyone may file; named peers verify."

5 The eight-state lifecycle

A binding moves through a state machine. The contract enumerates nine values; `None` is merely the default for a binding that does not exist, leaving eight reachable states.

#	State	Meaning
0	None	The binding does not exist (default).
1	Submitted	In active review; the pending window is open.
2	Canon	Canonized by peer consensus; in the public archive.
3	Expelled	Rejected by peer consensus (terminal).
4	Lapsed	Review window passed without a threshold; re-filable.
5	Contested	A canon binding under an active challenge.
6	Deprecated	Removed by challenge consensus (terminal).
7	Reaffirmed	Survived a challenge; re-confirmed as canon.
8	Queued	Parked behind a full active-review set; not yet in review.

The legal transitions are:

`Queued` → `Submitted`

`Submitted` → `Canon` | `Expelled` | `Lapsed`

`Lapsed` → `Submitted/Queued` (re-filed)

`Canon / Reaffirmed` → `Contested` → `Deprecated` | `Reaffirmed`

A newly opened binding enters `Submitted` if the active-review set has a free slot, otherwise it parks in `Queued` until promoted (Section 6). In `Submitted` it sits for a 30-day review window. Approvals reaching the canonize threshold move it to `Canon`; rejections that make canon arithmetically impossible settle it early; if neither happens before the window closes, anyone may call `markLapsed` to settle it. The early-settle and window-close paths share one rule (Section 8): an expel-quorum of rejections yields `Expelled` (terminal), otherwise `Lapsed` (re-filable with a fresh review round). Only `Canon` and `Reaffirmed` bindings appear in the public archive. A canon binding may later be challenged into `Contested`, resolving to `Deprecated` or `Reaffirmed` (Section 9). Expelled, Lapsed, and Deprecated bindings are not erased — they remain on-chain as part of the permanent record of what the network decided.

Window / cooldown (constant)	Scope	Duration
`PENDING_WINDOW` (review)	per binding	30 days
`CHALLENGE_WINDOW`	per binding	21 days
`CHALLENGE_COOLDOWN`	per peer	7 days
`RECHALLENGE_COOLDOWN`	per binding	30 days
`PROPOSAL_WINDOW` (taxonomy)	per node	30 days
`REVOKE_WINDOW`	per motion	14 days
`INACTIVITY_WINDOW` (peer GC)	per peer	30 days
`SUBMIT_COOLDOWN` (non-peer)	per address	10 minutes
`BOOST_COOLDOWN` (non-peer)	per address	10 minutes

6 The submission queue: open intake, bounded review

Open submission scales without limit; human review does not. If every submission entered active review immediately, a burst of filings — honest or hostile — could bury the peers under more open reviews than they could ever process, and the 30-day clock on each would start ticking regardless. The contract bounds the *active* review set instead.

```
reviewCapacity() = activePeerCount * REVIEW_SLOTS_PER_PEER
(REVIEW_SLOTS_PER_PEER = 4)
```

When a binding is opened, it enters `Submitted` only if `activeReviewCount < reviewCapacity()`; otherwise it parks in `Queued` with its review clock unset. As bindings resolve (canonized / expelled / lapsed) they free slots, and queued bindings are promoted to fill them:

- `boostQueued(id, topicId)` casts *one* boost on a queued binding to raise its priority. It is open to *anyone* — a boost only reorders the queue (the keeper promotes by priority) and never touches the consensus verdict, so it is open triage, not a vote. Two anti-spam guards bound it: one boost per wallet per binding, and a per-wallet `BOOST_COOLDOWN` (10 minutes) so a single identity cannot sweep-boost the whole queue; active peers are exempt from the cooldown, mirroring the public submit policy. Because a boost can only change the *order* in which already-queued evidence is reviewed — never what canonizes — the worst a sybil flood achieves is reordering the queue, which the permissionless drain (below) makes self-correcting.
- `promote(id, topicId)` moves the highest-priority queued binding into `Submitted` once a slot is free, starting its review clock. Promotion is *permissionless*: a keeper (the `consensus-keeper` edge function, Section 15) calls it in priority order, but anyone may call it, so the queue always drains even if the keeper is offline. Only ordering — never liveness — depends on the keeper.

This global bound, together with a per-address `SUBMIT_COOLDOWN` of 10 minutes on non-peer submitters, replaces an earlier per-submitter outstanding cap: a single identity can no longer flood the *active* set no matter how much it queues, and the review load carried by each peer stays proportional to the size of the peer set.

> **Self-healing by construction.** `activeReviewCount` falls every time a binding leaves `Submitted`, which immediately frees a slot for the next promotion. The queue therefore drains at exactly the rate the peers clear reviews, and it cannot deadlock: every path out of `Submitted` decrements the counter, and `promote` is callable by anyone.

7 Voting and threshold mathematics

Every threshold is a function of `activePeerCount`, the live size of the verified peer set, so the bar tracks the network as it grows and the system is never deadlocked when it is small. Every threshold carries a **floor of 1**, so a single Genesis peer can act without stalling.

For canonize, expel, and deprecate, the threshold is the ceiling of a percentage of the peer count:

$$t(n) \;=\; \max(1,\; \lceil\, n \cdot p \,\rceil)\,,$$

where n is the relevant peer count and p is the percentage below.

Action	Tier I	Tier II	Tier III
Canonize (approvals to accept)	60%	55%	51%
Expel (rejections to reject)		25% (all tiers)	
Deprecate (challenge votes)	65%	60%	55%

Canon is a true majority. Canonization clears more than half of the peer set at every tier (60 / 55 / 51%), so "Canon" reflects majority consensus, never a large minority. The asymmetry across tiers is intentional. The strongest evidence (Tier I: peer-reviewed and declassified) is the *hardest* both to admit (60%) and to remove once admitted (65%): the record demands a high bar to take its strongest claims seriously, and an even higher bar to retire them. Testimony (Tier III) is the easiest to admit (51%) but still requires a substantial majority (55%) to deprecate, so a contested first-person account is not casually discarded. Early expulsion of a still-pending record is tier-independent and deliberately low (25%): a record the network is clearly unwilling to accept should not have to wait out the full window.

Four further thresholds govern membership and structure. They are deliberately *decoupled* so that admitting a peer, ratifying a node, removing a peer, and retiring a node each clear an appropriate and distinct bar:

Gate	Formula	Meaning
Admission (nominee)	$\lfloor n/3 \rfloor + 1$	strictly more than 1/3
Taxonomy ratify	$\lfloor n/2 \rfloor + 1$	strict majority
Revoke a peer	$\lceil n/2 \rceil$	simple majority
Retire a node	$\lceil 2n/3 \rceil$	2/3 supermajority

The founding-bundle gate composes two of these: bundleThreshold(tier) = max($\lfloor n/2 \rfloor$ + 1, canonizeThreshold(tier)), so a founding record is never canonized on a cheaper vote than ordinary review would demand.

Double-voting is impossible: the contract records, per binding and per phase (review or challenge), whether a given address has already voted, and each re-filing or re-challenge bumps a round counter that isolates the new tally from the old. Peers may also cast review votes in batches (up to `MAX_REVIEW_BATCH = 50` per transaction) to process a queue economically.

8 Order-independence via peer-count snapshots

A subtle hazard in any threshold-on-a-percentage scheme is that the denominator moves: peers join and leave *during* a review window. If the verdict depended on when each vote happened relative to that churn, the same set of votes could canonize or not by accident of timing. The contract removes this dependence deliberately.

When a binding enters review (at submission or at promotion) it *snapshots* `activePeerCount` into `peerSnapshot`. The **canonize target** is then judged against that frozen snapshot, so a fixed number of approvals canonizes regardless of membership changes after the window opened. The same snapshot governs the **expel** bar.

Early settlement, however, asks a different question, and so it uses the *live* count. A reject vote triggers settlement only once canon has become arithmetically impossible — when, even if every peer who has not yet voted approved, approvals still could not reach the canonize target:

$$\text{canonizeTarget} + \text{rejects} > \texttt{activePeerCount}.$$

Judging "canon impossible" against the live electorate keeps it mutually exclusive with canonization under peer-set *growth*: late-joining peers can still rescue a record that a snapshot-based test would have prematurely declared dead. When settlement does fire, an expel-quorum of rejections ($\geq$ the snapshot's 25% bar) yields `Expelled`; otherwise the binding merely failed to attract a verdict and `Lapses`, re-filable later. The one residual effect of churn is benign: a heavy peer-set *shrink* can turn what would have been an expulsion into a lapse, so membership loss can never *terminally* expel otherwise-canonizable evidence — it can only send it back to the queue against the smaller electorate.

> **The guarantee.** Within a review round the outcome is independent of the order in which votes arrive: a binding canonizes the instant approvals reach the (snapshot) target, and is expelled early only once canon is genuinely impossible against the live set. Snapshot for the target, live count for "impossible" — the two are chosen so the two conclusions can never both be true at once.

9 Challenges: keeping the record revisable

Canon is never permanent by default. Any verified peer may challenge a `Canon` or `Reaffirmed` binding with `openChallenge(id, topicId)`, which moves it to `Contested` and starts the 21-day challenge window. The challenger's own vote is cast automatically as the first challenge vote. Two cooldowns keep the mechanism from becoming a weapon: a *per-peer* `CHALLENGE_COOLDOWN` (7 days) limits how often one peer can open challenges, and a *per-binding* `RECHALLENGE_COOLDOWN` means a record cannot be re-contested until its last challenge window has closed plus 30 days — so a rotating cast of peers cannot keep evidence perpetually in limbo.

During the window, peers vote with `castChallengeVote(id, topicId, support)` — either supporting the challenge or defending the record. The resolution is a race between a threshold and a clock:

- If support votes reach the tier-dependent deprecate threshold, the binding becomes `Deprecated` immediately.
- If the 21-day window closes without that threshold being met, the binding is `Reaffirmed` — it survived, and is now a record that has been tested and held.

Either outcome is settled with `finalizeChallenge(id, topicId)`. Crucially, deprecation is judged against the *live* peer set, not a snapshot: destroying canon must always clear a supermajority of the network *as it stands now*, so peers admitted during the challenge window raise the bar, never dilute it. Because a reaffirmed binding can itself be challenged again later (after its cooldown), no record is ever beyond re-examination; it is only ever "standing for now."

10 The peer registry: membership without an administrator

The set of peers who may judge evidence is governed entirely on-chain, by the peers themselves.

Genesis and seed phase. The system launches from a Genesis peer set supplied at deployment (the quorum floor of 1 lets even a one-peer network act). During an initial seed phase the owner may seed further peers directly with `addPeer`, but *only* while `activePeerCount < seedPhaseK`; the moment the active set reaches `seedPhaseK`, ordinary nominations open (`nominationsOpen()`) and the owner's seeding power ends.

Steady state. A newcomer is brought in by an existing peer via `nominatePeer(nominee, handle)` (the handle is at most 64 bytes). Other peers then endorse the nominee with `endorseNominee`; when endorsements reach the admission gate $\lfloor n/3 \rfloor + 1$, the contract verifies the nominee *automatically* — no administrator approves the final step. A nomination that never reaches its gate can be cleared after the proposal window by the permissionless `lapseNominee`, freeing the address to be nominated again with a fresh endorsement round.

Removal. A peer can be removed by the same community that admits them: `motionRevoke` opens a motion and `voteRevoke` accrues votes, with removal at a majority $\lceil n/2 \rceil$. A stale motion that never passed is cleared after `REVOKE_WINDOW` by `cancelStaleRevocation`, so no peer is left indefinitely under an open cloud. A peer-floor invariant forbids removing the last peer.

Ownership. The owner role exists only for the seed phase and for emergency `pause`/`unpause`; it can never force a record in, force a peer out, or override a vote. Ownership transfers in a safe two-step `proposeOwner` → `acceptOwnership` handshake (cancellable), and once the seed phase is complete and the contract is live the owner may `renounceOwnership` to drop the role entirely.

> **Identity is a wallet plus a handle.** A peer is a wallet address carrying a human-readable handle. The system makes no claim that a wallet is a verified legal identity — but every judgement is permanently tied to the wallet that signed it, which is what accountability requires here.

11 Capture resistance and the force-renounce backstop

An ownerless system is only as good as its resistance to a coalition quietly *becoming* the owner. The registry is designed around an explicit capture boundary.

The admission gate is the capture line. Admission requires *strictly* more than one third of peers ($\lfloor n/3 \rfloor + 1$). The strict inequality is load-bearing: with a mere $\lceil n/3 \rceil$ a coalition of exactly $n/3$ (when n is divisible by 3) could admit its own members and bootstrap itself to a majority. The strict gate closes that zero-margin case, so any coalition holding $\leq 1/3$ of the peers cannot admit anyone — nor ratify or retire a node — without honest cooperation. This is the familiar BFT boundary: the system is safe as long as more than two thirds of peers are honest.

Creating and destroying canon taxonomy are both kept above easy reach. Ratifying a node is a strict majority ($\lfloor n/2 \rfloor + 1$) and retiring one is a 2/3 supermajority ($\lceil 2n/3 \rceil$). Decoupling the ratify gate from the lower admission gate means a sub-majority faction can no longer spawn nodes that a supermajority must then clean up; the create/retire gap narrows from $1/3 \rightarrow 2/3$ to $1/2 \rightarrow 2/3$, and the retire bar sits safely above the capture line.

The force-renounce backstop. The owner's only powers are pause and seed-phase `addPeer`, and `renounceOwnership` is gated on the contract not being paused — so a malicious or compromised owner could, in principle, pause forever and brick the network with no peer recourse. One

motion closes that gap. `motionForceRenounce` / `voteForceRenounce` let a 2/3 peer supermajority (the same bar used to destroy canon governance) strip the owner entirely *and* unpause the contract. These two functions deliberately carry *no* "when-not-paused" guard — their whole purpose is to act precisely when a captured owner has paused everything. When they pass, ownership passes to no one: `owner` is set to the zero address and the network continues fully self-governed.

> **What this buys.** "Owned by no one" is not merely a promise to renounce. Even an owner who refuses to renounce, and who pauses the contract to entrench, can be evicted by two thirds of the peers — the same honest majority the whole BFT model already assumes. The keys cannot be held against the network's will.

12 Peer liveness and garbage collection

Every threshold divides by `activePeerCount`. If that denominator counted peers who had silently abandoned the network, the bars would drift out of reach and a live, willing minority could be unable to reach any quorum. The contract keeps the denominator honest with an objective liveness clock.

Each peer carries a `lastActive` timestamp, set on admission and refreshed by every consensus vote (review or challenge) and by an explicit `heartbeat()` — the latter for a peer who is present and willing but has nothing to review right now. A peer idle past `INACTIVITY_WINDOW` (30 days) can be removed by anyone via the permissionless `pruneInactivePeer` — in practice the `consensus-keeper` (Section 15) runs it automatically on a schedule, but because the call is permissionless the keeper is a convenience, not a dependency. Because inactivity is objective and on-chain, no vote is needed (unlike the subjective misbehaviour revoke), and the prune can never drop the active set below the seed-phase floor, so garbage collection can neither strand nor capture the network.

The same permissionless-after-the-window discipline cleans up every other transient state: `lapseNominee` clears a dead nomination, `cancelStaleRevocation` clears a dead revoke motion, `lapseProposal` clears a dead taxonomy proposal, `cancelStaleRetire` clears a dead retire motion, and `markLapsed` / `finalizeChallenge` settle a binding whose window has closed. None of these require privilege; the contract never depends on a caretaker to make progress.

13 Taxonomy retirement

A ratified node is never deleted — the log is immutable — but a strong peer supermajority can *retire* a node so it drops out of the public taxonomy. Retirement mirrors the revocation motion: `motionRetireNode(id)` opens it (the motioner is retire-vote one), `voteRetireNode(id)` accrues votes, and at $\lceil 2n/3 \rceil$ the node flips to `Retired` and is removed from the public lists. A stale motion is cleared by `cancelStaleRetire` after the window.

The mechanism enforces "no empty pillars" from the other direction. Only **topics** are retired directly; `motionRetireNode` reverts on a pillar. A pillar is retired *automatically*, in the same transaction, when its *last* topic is retired — so a pillar can never sit ratified with zero topics. (Symmetrically, a topic proposal that would attach to a pillar retired while the proposal was in flight is not ratified; it is left to lapse, so a simple-majority proposal can never partially undo a 2/3 retirement.) The off-chain indexer flips both rows to `retired` as it handles each `NodeRetired` event.

14 Attestations: every vote is a signed artefact

On-chain tallies record *that* a vote happened. To record *what* a peer attested, in a form a human can read and any party can verify independently, every vote is also carried as an EIP-712 typed-data signature in the off-chain layer.

The signing domain binds the signature to this exact contract and chain:

```
domain = { name:  "EvidenceConsensus", version:  "1",
    chainId, verifyingContract }
```

and the message type ties the attestation to a specific binding:

```
Attestation {
   evidenceId :  string
   topicId :  string
   peerAddr :  address
   phase :  string  // review, challenge, or taxonomy
   verdict :  string  // approve/reject/challenge/defend/endorse
   note :  string
}
```

Because `topicId` is part of the signed message, a signature is cryptographically bound to the precise (evidence × topic) binding being voted on — a vote cannot be replayed against a different filing. The signature is produced through the peer's wallet (`eth_signTypedData_v4`) and verified by the `verify-attestation` edge function, which recovers the signer, confirms it is an active peer, and recomputes the content hash before recording the attestation. The off-chain store keeps exactly one attestation per `(binding_id, peer_addr, phase)`, mirroring the contract's no-double-voting rule. Critically, this check need not be trusted: any visitor can re-run the identical EIP-712 recovery in their own browser over the public `(domain, types, message, signature)` and confirm that the *named peer* — not the platform — authored the vote.

Taxonomy endorsements are signed too. A peer endorsing a pillar/topic proposal (`endorseNode`) is the same accountable, signed act as a review or challenge vote, so it is recorded as a first-class attestation in the `taxonomy` phase with verdict `endorse`, tied to the proposal's founding binding (every proposal ships with one). `verify-attestation` confirms it against the on-chain `NodeEndorsed` event before recording it. A taxonomy *rejection* is admitted as a signed dissent (verdict `reject`) with no on-chain counterpart — the contract has no reject-node call — so it skips tx-hash verification. Either way the signed log is purely a record: the vote-counting triggers act only on the `review` and `challenge` phases, so a taxonomy attestation never moves a binding's tallies, and the on-chain `endorseNode` gate remains the sole consensus mechanism.

15 The off-chain projection

The Supabase layer is a single, rebuildable projection. Its main tables mirror the on-chain concepts:

Table	Holds
`evidence`	Canonical content + `content_hash`.
`bindings`	One row per (evidence, topic): status, tallies, `binding_hash`.
`pillars` / `topics`	Taxonomy nodes (proposed → ratified → retired), joined by `node_hash`.
`attestations`	One per `(binding_id, peer_addr, phase)`.
`chain_events` / `chain_event_cursor`	Indexer input and progress.
`tamper_alerts`	Evidence content-hash mismatches *and* taxonomy node_hash / meta_hash drift.

Four edge functions operate the layer: `chain-indexer-evidence` (reads chain events and reconciles the projection by `binding_hash` and `node_hash`), `verify-attestation` (verifies signatures and recomputes content hashes on write), `audit-content-hash` (re-hashes the archive *and* the taxonomy on a schedule to detect drift — it recomputes each evidence `content_hash` and each pillar/topic `node_hash` + `meta_hash`, opening a `tamper_alert` on any mismatch and auto-resolving one that matches again), and `consensus-keeper` (described below). Vote counts are applied through atomic database functions, `apply_review_counts` and `apply_challenge_counts`, so concurrent votes cannot corrupt a tally. Write endpoints are throttled by prefix in an `edge_rate_limit` table (`ev_insert:` for evidence, `tax_insert:` for taxonomy). Because the schema seeds nothing, the entire taxonomy and archive are produced at runtime by peers, never shipped as a baseline.

The keeper. `consensus-keeper` is a `pg_cron`-scheduled edge function (every five minutes) that drives the two on-chain maintenance jobs the contract cannot trigger for itself: it prunes peers idle past `INACTIVITY_WINDOW` (`pruneInactivePeer`, oldest-idle first, stopping at the seed-phase floor) and promotes queued bindings into review in public-priority order (`promote`, until `reviewCapacity` is full). It holds a signing key (`KEEPER_PRIVATE_KEY`) to send these transactions, but it grants *no* authority: both calls are permissionless and objectively gated on-chain, so the keeper can do nothing any address could not already do, and anyone may run the same jobs if it goes offline. It is a convenience that automates liveness, never a privileged actor.

16 A note on residual risk: open-submission front-running

Honesty about limits is part of the design. `submitEvidence` is permissionless, and on a public chain an evidence id (the Supabase UUID) is visible in the mempool before it confirms. An adversary can therefore front-run a submission with the *same* id but a different content hash, reverting the victim ("already submitted") and binding the id to junk content. This is bounded, not catastrophic:

- the off-chain `audit-content-hash` job flags any on-chain hash that does not match the canonical payload, so the junk is detected;
- UUIDs are free, so the victim simply re-mints and resubmits;
- nothing enters the public archive without passing peer review, and taxonomy proposals are additionally peer-gated — any squatted-but- unratifiable node is freed by the permissionless `lapseProposal` after the window.

A commit-reveal submission scheme would close the window entirely, at the cost of a second transaction per submission; it is deliberately omitted here. Documenting the residual rather

than hiding it is the same instinct that animates the whole system: the gate is a published rule, and so are its edges.

17 How the engineering serves the philosophy

Every mechanism above exists to make a stated principle enforceable rather than merely promised.

Principle (the essay)	**Mechanism (this paper)**
Filed against a source	Content-only `contentHash`, recomputed in three places; mismatches raise tamper alerts.
Judged in public by named peers	On-chain votes plus EIP-712 attestations signed by named, wallet-verified peers; re-verifiable by anyone, no anonymous moderators.
Canon is a majority, not a faction	True-majority canonize thresholds (60 / 55 / 51%); order-independent via peer-count snapshots.
Democratized method, open intake	Permissionless `submitEvidence` with a bounded review queue; an administrator-free registry that admits and removes peers by consensus.
Evidence is not proof	Tiered evidence with asymmetric thresholds; "Canon" is a quorum judgement, never a truth flag.
Evolving consensus reality	Peer-grown Pillar → Topic taxonomy; a perpetual challenge route; reaffirm-or-deprecate revision; supermajority retirement.
Owned by no one, seizable by no one	A single open contract as sole authority; two-step transfer, renounce, and a 2/3 force-renounce that evicts a captured owner.
Capture-resistant	A strict 1/3+1 admission gate (BFT honest-supermajority boundary) and a liveness clock that keeps the quorum denominator honest.

The contract is the place where the philosophy stops being a description of intentions and becomes a description of what the system can and cannot do. "Anyone may file" is a function anyone can call. "Named peers verify" is a signature that recovers to a registered address. "A majority stands behind it" is a counted vote against a frozen denominator. "Owned by no one" is a key that two thirds of peers can take from any hand. "Revised forever" is a state machine with no terminal lock on canon. The mechanism is the argument that the values are meant — and the companion essay, *Disclosure: A Labour of Love*, is the argument for why they are worth meaning in the first place.

> **Open source.** None of this has to be taken on trust — every claim in this paper is checkable against the code. The smart contract, its Lens sidecar, the edge functions, the frontend, and the sources of both whitepapers live in the public repository:
>
> https://github.com/DisclosureG/the-disclosure-platform